Language Definition

1 Abstract Syntax

1.1 Values

$$\begin{array}{rcl} v & ::= & n \\ & | & b \\ & | & C(v_1, \ldots, v_n) \end{array}$$

where $n \in \mathbb{Z}$, $b \in \{\texttt{true}, \texttt{false}\}$, and C is a constructor name.

1.2 Types

$$\begin{array}{rcl} T & ::= & \texttt{int} \\ & | & \texttt{bool} \\ & | & V \end{array}$$

Variant types are labeled sums:

$$V ::= \langle C_1 : \overline{T_1}, \ldots, C_n : \overline{T_n} \rangle$$

where each $\overline{T_i}$ denotes a (possibly empty) sequence of types and C_i are constructor names.

1.3 Expressions

$$\begin{array}{rcl} e & ::= & v \\ & | & e_1 \ \texttt{op} \ e_2 \\ & | & \texttt{!} \ e \\ & | & x \end{array}$$

where $\texttt{op} \in \{+, -, *, /, \wedge, \vee, ==, <, >\}$ and x is a variable name.

1.4 Statements

$$\begin{array}{rcl} s & ::= & \texttt{var} \ x : T \\ & | & x := e \\ & | & \texttt{if} \ e \ \{s^*\} \\ & | & \texttt{if} \ e \ \{s^*\} \ \texttt{else} \ \{s^*\} \\ & | & \texttt{print} \ e \\ & | & \texttt{variant} \ V = C_1(\overline{T_1}) \mid \cdots \mid C_n(\overline{T_n}) \end{array}$$

2 Typing Rules

$$\Gamma \vdash e : T$$

$$\Delta \vdash C : (\overline{T}) : V$$

2.1 Base Types

$$\frac{}{\Gamma \vdash n : \texttt{int}}\ \text{T-Int} \qquad \frac{}{\Gamma \vdash \texttt{true} : \texttt{bool}}\ \text{T-Bool-True} \qquad \frac{}{\Gamma \vdash \texttt{false} : \texttt{bool}}\ \text{T-Bool-False}$$

2.2 Variables

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T}\ \text{T-Var-Use}$$

2.3 Binary Operators

$$\frac{\Gamma \vdash e_1 : \texttt{int} \qquad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 + e_2 : \texttt{int}}\ \text{T-Arithmetic} \qquad \frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \texttt{bool}}{\Gamma \vdash e_1 \wedge e_2 : \texttt{bool}}\ \text{T-Logical}$$

$$\frac{\Gamma \vdash e_1 : \texttt{int} \qquad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 < e_2 : \texttt{bool}}\ \text{T-Comparison} \qquad \frac{\Gamma \vdash e_1 : T \qquad \Gamma \vdash e_2 : T}{\Gamma \vdash e_1 == e_2 : \texttt{bool}}\ \text{T-Equality}$$

2.4 Unary Operators

$$\frac{\Gamma \vdash e : \texttt{bool}}{\Gamma \vdash \texttt{not}\ e : \texttt{bool}}\ \text{T-Not}$$

2.5 Variants

$$\frac{\Gamma \vdash e_1 : T_1, \cdots, e_n : T_n \qquad \Delta \vdash C : (T_1, \ldots, T_n) : V}{\Gamma, \Delta \vdash C(e_1, \ldots, e_n) : V}\ \text{T-Variant-Constructor}$$

The constructor C must be declared as part of variant V with argument types $(T_1, \ldots, T_n)$.

3 Operational Semantics

The operational semantics are given as a big-step evaluation relation:

$$\langle e, \sigma \rangle \Downarrow v$$

3.1 Base Values

$$\frac{}{\langle n, \sigma \rangle \Downarrow n}\ \text{E-Int} \qquad \frac{}{\langle \texttt{b}, \sigma \rangle \Downarrow \texttt{b}}\ \text{E-Bool}$$

3.2 Variables

$$\frac{\sigma(x) = v}{\langle x, \sigma \rangle \Downarrow v}\ \text{E-Var}$$

3.3 Operations

$$\frac{\langle e_1, \sigma\rangle \Downarrow n_1 \qquad \langle e_2, \sigma\rangle \Downarrow n_2}{\langle e_1 + e_2, \sigma\rangle \Downarrow n_1 + n_2}$$

$$\frac{\langle e_1, \sigma\rangle \Downarrow b_1 \qquad \langle e_2, \sigma\rangle \Downarrow b_2}{\langle e_1 \wedge e_2, \sigma\rangle \Downarrow b_1 \wedge b_2}$$

$$\frac{\langle e, \sigma\rangle \Downarrow b}{\langle \texttt{!}e, \sigma\rangle \Downarrow \neg b}$$

$$\frac{\langle e_1, \sigma\rangle \Downarrow n_1 \qquad \langle e_2, \sigma\rangle \Downarrow n_2}{\langle e_1 < e_2, \sigma\rangle \Downarrow (n_1 < n_2)}$$

$$\frac{\langle e_1, \sigma\rangle \Downarrow v_1 \qquad \langle e_2, \sigma\rangle \Downarrow v_2}{\langle e_1 == e_2, \sigma\rangle \Downarrow (v_1 == v_2)}$$

3.4 Variants

$$\frac{\langle e_1, \sigma\rangle \Downarrow v_1 \qquad \cdots \qquad \langle e_n, \sigma\rangle \Downarrow v_n}{\langle C(e_1, \ldots, e_n), \sigma\rangle \Downarrow C(v_1, \ldots, v_n)}$$

4 Statements

Statement execution is defined as:

$$\langle s, \sigma\rangle \Downarrow \sigma'$$

$$\frac{\langle e, \sigma\rangle \Downarrow v}{\langle x := e, \sigma\rangle \Downarrow \sigma[x \mapsto v]}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{true}}{\langle \texttt{if}\ \ e\{s^*\}, \sigma\rangle \Downarrow \sigma'}$$

5 Conclusion

This document formally specifies the abstract syntax, typing rules, and operational semantics of the language as implemented in the provided C++ source files.